SCHOOL OF TECHNOLOGY	WOXSEN UNIVERSITY

DATABASE MANAGEMENT SYSTEMS

PROJECT-BASED LEARNING (PBL) REPORT

Small Database Application Development using Java / Python

PROJECT TITLE

DESIGN AND IMPLEMENTATION OF A DATABASE MANAGEMENT SYSTEM FOR BANK ACCOUNT AND TRANSACTION MANAGEMENT SYSTEM
(Project 55)

Program	AI-ML
Section	AI-ML RHINOS
Faculty	Prof. Sahithi Chennamadhavun
Date of Submission	14 October 2026
Academic Year	2026 – 27

Submitted by

Roll Number	Student Name	Mobile	Email ID
25WU0102140	M. Srilakshmi Reddy	8918595029	reddy14srilakshmi@gmail.com

ACKNOWLEDGEMENT

I express my sincere gratitude to **Woxsen University, School of Technology**, for providing me with the opportunity to undertake this Project-Based Learning (PBL) activity as part of the **Database Management Systems (DBMS)** course.

I thank my course faculty, **Prof. Sahithi Chennamadhavun**, for the guidance, suggestions, and support throughout the design and development of this project. The guidance helped me understand the practical application of database concepts such as ER modelling, relational schema design, normalization, SQL, constraints, queries, and database application development.

This PBL activity has strengthened my understanding of DBMS concepts and provided valuable experience in applying them to a real-world problem.

UNDERTAKING

I hereby declare that the project entitled “Design and Implementation of a Database Management System for Bank Account and Transaction Management System”, submitted as part of the Project-Based Learning activity for the Database Management Systems (DBMS) course, is my individual work carried out under the guidance of the course faculty.

I undertake that the project report, database design, SQL scripts, application source code, results, and supporting materials submitted by me have been prepared for academic purposes. Wherever information, ideas, tools, datasets, or other resources have been referred to, appropriate acknowledgement has been provided.

I further undertake that I am responsible for the correctness and authenticity of the work submitted and that I have followed the academic and project guidelines prescribed by the University.

Signature

Date : 14 October 2026

Place : Hyderabad

Name: M. Srilakshmi Reddy

Roll Number: 25WU0102140

Program: AI-ML

TABLE OF CONTENTS

Contents	Page No.
Abstract	8
LIST OF TABLES	5
LIST OF FIGURES	6
LIST OF ACRONYMS	7
1. PROBLEM STATEMENT, SOLUTION STRATEGY AND ER DIAGRAM	9
1.1 Problem Statement	9
1.2 Objectives	9
1.3 Scope and Assumptions	10
1.4 Proposed Solution Strategy	10
1.5 Functional Requirements	11
1.6 Entity-Relationship Diagram (ERD)	11
1.7 Entity and Relationship Summary	12
2. LOGICAL DATABASE DESIGN	14
2.1 Relational Schema	14
2.2 Keys and Integrity Constraints	15
2.3 Functional Dependencies	17
2.4 Normalization up to Third Normal Form (3NF)	17
2.5 Data Dictionary	18
2.6 Implementation of Logical Design: DDL and Sample Data	20
3. APPLICATION DEVELOPMENT USING LOGICAL DATABASE DESIGN	23
3.1 Application Overview	23
3.2 Development Environment and Database Connectivity	23
3.3 Application Modules and Database Mapping	24
3.4 CRUD Operations	24
3.5 Business Rules, Validation and Error Handling	24
3.6 Reports and Database Operations	25
3.7 Application Screens	26
3.8 Testing and Results	31
4. CONCLUSION	34
4.1 Limitations and Future Enhancements	34
5. REFERENCES	35
6. APPENDICES	36

LIST OF TABLES

Table No.	Table Title	Page No.
Table 1	Scope and Assumptions	10
Table 2	Proposed Solution Strategy	10
Table 3	Functional Requirements	11
Table 4	Entity and Relationship Summary	12
Table 5	Relational Schema	14
Table 6	Keys and Integrity Constraints	15
Table 7	Normalization Status up to 3NF	18
Table 8	Data Dictionary	18
Table 9	Database Implementation Environment	20
Table 10	Application Development Environment	23
Table 11	Application Module-to-Database Mapping	24
Table 12	CRUD Operations	24
Table 13	Business Rules, Validation and Error Handling	25
Table 14	Reports and Database Operations	25
Table 15	Test Cases	31

LIST OF FIGURES

Figure No.	Figure Title	Page No.
Figure 1	Entity-Relationship Diagram of the Proposed System	12
Figure 2	The 14 base tables in the bank_db database (MySQL)	22
Figure 3	Staff Sign-in Screen	26
Figure 4	Manager Dashboard with Live Totals and Branch Summary	27
Figure 5	Account Search and Display	27
Figure 6	Cash Deposit Posted to an Account (Data Entry)	28
Figure 7	Withdrawal Beyond the Balance Rejected by the Database	28
Figure 8	Atomic Transfer Between Two Accounts	29
Figure 9	Loan Schedule After an EMI Repayment (Update)	29
Figure 10	Account Statement Report with Running Balance	30
Figure 11	Rule Tester: Illegal Operations Refused and Rolled Back	31
Figure 12	Trigger Rejection and Unchanged Balances in MySQL	32

LIST OF ACRONYMS

Acronym	Expansion
3NF	Third Normal Form
AI	Auto Increment
API	Application Programming Interface
BCNF	Boyce-Codd Normal Form
CRUD	Create, Read, Update, Delete
DBMS	Database Management System
DDL	Data Definition Language
DF	Default
EMI	Equated Monthly Instalment
ER	Entity-Relationship
ERD	Entity-Relationship Diagram
FK	Foreign Key
FR	Functional Requirement
GUI	Graphical User Interface
HTTP	HyperText Transfer Protocol
IDE	Integrated Development Environment
IFSC	Indian Financial System Code
INR	Indian Rupee
JSON	JavaScript Object Notation
JWT	JSON Web Token
KYC	Know Your Customer
NEFT	National Electronic Funds Transfer
NN	Not Null
PBL	Project-Based Learning
PDF	Portable Document Format
PK	Primary Key
REST	Representational State Transfer
SMS	Short Message Service
SQL	Structured Query Language
UNF	Unnormalised Form
UPI	Unified Payments Interface
UQ	Unique

ABSTRACT

Banks record thousands of customer, account and transaction entries daily, and manual or loosely structured record keeping leads to balance errors, duplicated identifiers, unbalanced transfers and weak audit trails. This project aims to design and implement a relational database system for bank account and transaction management that enforces these business rules inside the database itself. The conceptual model was drawn as an entity-relationship diagram and converted into a fourteen-table logical schema normalised to third normal form, with primary key, foreign key, unique, check and default constraints. It was implemented in MySQL with three triggers, two stored procedures, seven reporting views and five indexes, and loaded with realistic multi-branch sample data. A Node.js and Express application with role-based staff login sits on top of the database and provides customer and account maintenance, deposits, withdrawals, atomic transfers, loan disbursement with EMI schedules, repayments, statements and a rule tester. Sixteen test cases show that illegal operations such as overdrawing, posting to frozen or closed accounts and unbalanced transfers are rejected and rolled back, while every stored balance equals its ledger. The system demonstrates a dependable, auditable core for small-scale retail banking.

KEYWORDS

Keywords: Relational Database, ER Diagram, Normalization (3NF), MySQL, Triggers and Stored Procedures, Banking Transactions

CHAPTER 1

PROBLEM STATEMENT, SOLUTION STRATEGY AND ER DIAGRAM

1.1 Problem Statement

A retail bank serves thousands of customers through several branches, and every deposit, withdrawal, transfer, service charge and loan instalment changes the money held on someone's behalf. When these records are kept in spreadsheets, paper registers or loosely structured files, the same customer or account can be entered twice, account numbers and transaction references can be duplicated, a withdrawal can be accepted although it takes the balance below the permitted minimum, and the two halves of a transfer can be posted unevenly. Tellers, loan officers and branch managers then have no single trustworthy source from which to answer balance, statement and loan-outstanding questions.

The affected stakeholders are account holders, who depend on correct balances and statements; tellers and loan officers, who post transactions and collect repayments; and branch managers, who need branch-level summaries and an audit trail. The manual process is slow, depends on the care of the person posting, cannot stop a rule violation at the moment it is made, and turns reconciliation into a periodic clean-up instead of a property of the data. A relational database that enforces the banking rules itself, together with a small application that uses it, is therefore needed so that invalid operations are refused before they reach the ledger and every balance can be traced back to its postings.

1.2 Objectives

The objectives of the project are to:

- Design an entity-relationship model for retail banking that covers customers, branches, account types, accounts, joint holders, transactions, transfers, beneficiaries, service charges, loans, loan schedules, repayments and statements.
- Develop a relational schema of 14 tables, normalised to Third Normal Form, with primary key, foreign key, unique, not-null, check and default constraints.
- Implement the business rules inside MySQL through 3 triggers and 2 stored procedures so that illegal postings (non-positive amounts, overdrawing, frozen or closed accounts and unbalanced transfers) are rejected and rolled back.

- Automate loan EMI scheduling, repayment allocation and overdue marking, and generate account statements and branch reports through 7 views and 12 verified queries.
- Validate the design through a working web application with role-based login and 16 positive and negative test cases, confirming zero balance mismatches.

1.3 Scope and Assumptions

Table 1 defines what the project covers, what it deliberately leaves out, and the assumptions on which the design rests.

Table 1. Scope and Assumptions

Item	Description
In Scope	Customer, branch and account-type master data; account opening with primary and joint holders; deposits, withdrawals, transfers between accounts, beneficiaries and service charges; personal, vehicle, education and home loans with EMI schedules, repayments and overdue tracking; account statements; branch summary and audit reports; staff login with Teller, Loan Officer and Manager roles.
Out of Scope / Exclusions	Mobile banking apps, real-time payment gateways (UPI, NEFT, card networks), multi-currency accounts, periodic interest-posting runs, customer self-service login, regulatory and KYC verification, and settlement of transfers to accounts held at other banks.
Assumptions	Single currency (INR); one bank with four branches; account balances change only through ledger postings; a joint account has one primary holder and any number of joint holders; EMIs follow the reducing-balance formula and are paid in full from the linked account; the database server is MySQL 8; staff use only the role they are issued.

1.4 Proposed Solution Strategy

The solution stores all banking facts in a normalised relational database and keeps the rules next to the data, following the relational model [1]. A thin application layer only collects input and displays results, so every access path, whether the web application, a script or a manual SQL session, meets the same rules. Table 2 summarises the proposed solution.

Table 2. Proposed Solution Strategy

Solution Element	Project-Specific Description
Primary Users / Roles	Teller: customers, accounts, cash postings, transfers, beneficiaries, statements. Loan Officer: loan creation and repayment collection. Branch Manager: all functions, plus the audit log and account-status control (freeze, close).
Major Application Modules	Login, Dashboard, Customers, Accounts, Transactions, Transfers, Statements, Loans, and Rule Tester.
Proposed Workflow	Staff sign in; a customer is registered; an account is opened (with joint holders if needed) and funded; deposits, withdrawals and transfers post to the ledger; loans are disbursed into the account with an EMI schedule and repaid instalment by instalment; statements and reports are generated from views.
Important Business Rules	Amounts must be positive; a debit may not take the balance below the account type's minimum; only ACTIVE accounts accept postings; a transfer needs two different accounts and equal debit and credit legs; account numbers, e-mails and ID proofs are unique; an instalment is closed only when fully paid; failed operations leave no partial data.

Solution Element	Project-Specific Description
Expected Reports / Outputs	Account statement with running balance, branch summary, loan outstanding and overdue instalments, repayment history, transfer register, transaction audit and monthly deposit/withdrawal totals.

1.5 Functional Requirements

Table 3 lists the functional requirements. Priority reflects how central each requirement is to a correct banking ledger.

Table 3. Functional Requirements

Req. ID	Functional Requirement	Remarks / Priority
FR-01	Register, search and update customer profiles with unique e-mail and ID proof.	High
FR-02	Open an account for one or more holders at a branch with a chosen account type and optional opening deposit.	High
FR-03	Post deposits and withdrawals; reject non-positive amounts, overdrawing and postings to non-ACTIVE accounts.	High
FR-04	Transfer money between two accounts as one atomic unit with equal debit and credit legs.	High
FR-05	Maintain beneficiaries (internal and external) for each customer.	Medium
FR-06	Apply service charges (SMS, ATM, cheque book, penalty, maintenance) as ledger postings.	Medium
FR-07	Create loans with disbursement and an automatic reducing-balance EMI schedule.	High
FR-08	Record repayments against an instalment, mark it PAID, close the loan after the last instalment.	High
FR-09	Mark unpaid instalments past their due date as OVERDUE.	Medium
FR-10	Generate account statements for a date range with opening, closing balance and running balance.	High
FR-11	Provide branch summary, loan outstanding, transfer, repayment and audit reports.	Medium
FR-12	Authenticate staff and restrict each role to its permitted functions.	High
FR-13	Show every database rejection to the user exactly as the database reports it, and leave data unchanged.	High

1.6 Entity-Relationship Diagram (ERD)

Figure 1 shows the approved entity-relationship diagram, generated from the implemented schema so that entity names, keys and cardinalities match the database exactly. The notation is crow's foot: a line ending in three prongs is the 'many' side, a bar is 'one', and a circle marks optional participation. The ER model [2] has 14 entities. TRANSFER, SERVICE_CHARGE and LOAN_REPAYMENT are specialisations of TRANSACTION, and ACCOUNT_HOLDER resolves the many-to-many relationship between CUSTOMER and ACCOUNT. Table 4 lists the entities and their relationships.

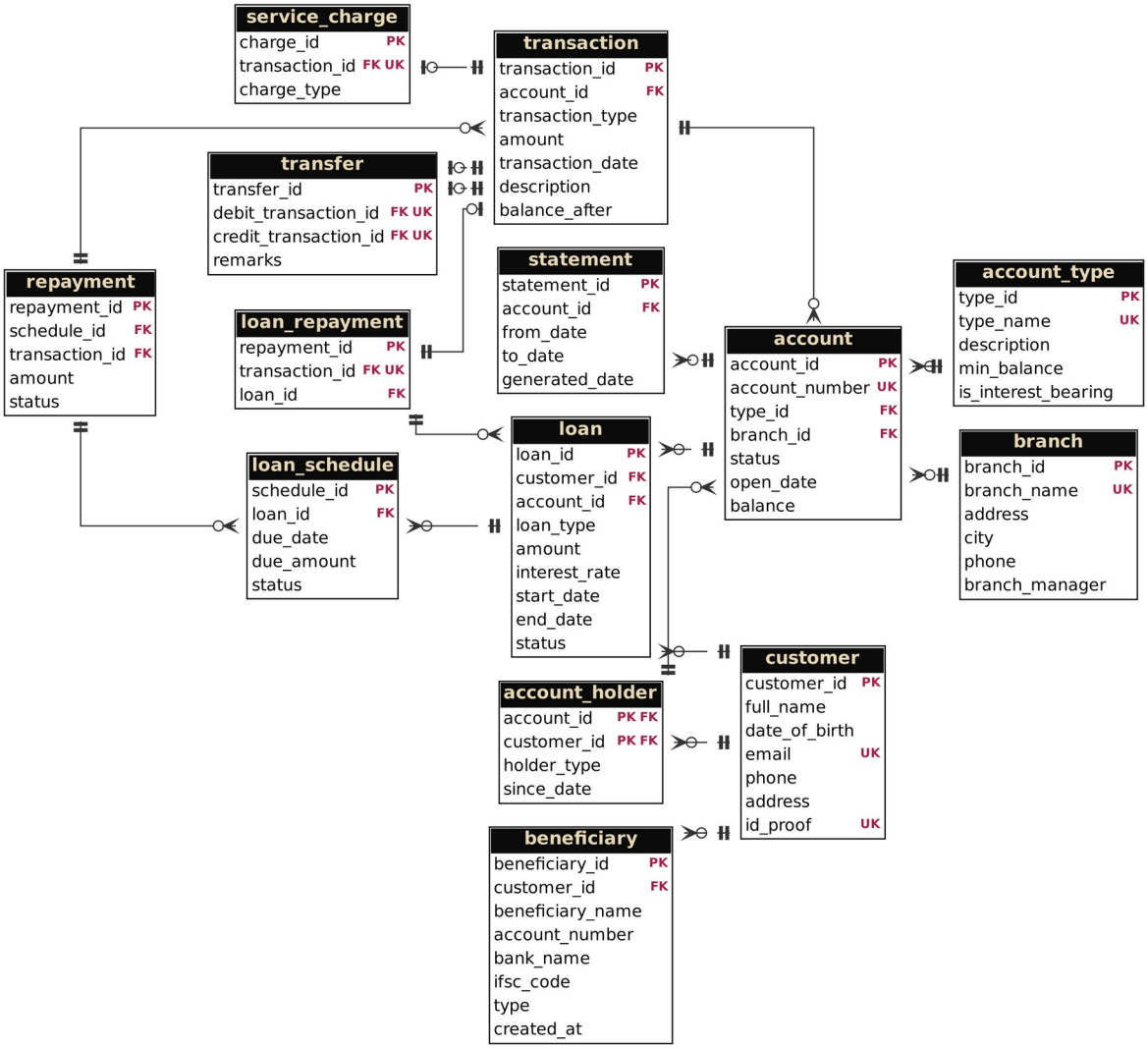


Figure 1. Entity-Relationship Diagram of the Proposed System

1.7 Entity and Relationship Summary

Table 4 summarises the entities and the relationships between them.

Table 4. Entity and Relationship Summary

Entity / Relationship	Major Attributes / Cardinality	Purpose / Rule
BRANCH	branch_id (PK), branch_name, address, city, phone, branch_manager	Bank branch; 1 branch : N accounts
ACCOUNT_TYPE	type_id (PK), type_name, min_balance, is_interest_bearing	Savings, Current, Salary, Fixed Deposit rules; 1 : N accounts
CUSTOMER	customer_id (PK), full_name, date_of_birth, email, phone, id_proof	Person who banks here; e-mail and ID proof unique
ACCOUNT	account_id (PK), account_number, type_id (FK), branch_id (FK), status, open_date, balance	Holds money; balance maintained only by trigger
ACCOUNT_HOLDER	(account_id, customer_id) (PK), holder_type, since_date	Resolves customer M : N account; PRIMARY or JOINT holder
TRANSACTION	transaction_id (PK), account_id (FK), transaction_type, amount, transaction_date, balance_after	Every money movement; supertype of transfer, charge, loan repayment

Entity / Relationship	Major Attributes / Cardinality	Purpose / Rule
TRANSFER	transfer_id (PK), debit_transaction_id (FK), credit_transaction_id (FK), remarks	One transfer = one debit leg + one credit leg
SERVICE_CHARGE	charge_id (PK), transaction_id (FK), charge_type	Fee posted as a transaction
BENEFICIARY	beneficiary_id (PK), customer_id (FK), beneficiary_name, account_number, ifsc_code, type	Saved payee; unique per customer, account and IFSC
STATEMENT	statement_id (PK), account_id (FK), from_date, to_date, generated_date	Log of generated statements; period must be valid
LOAN	loan_id (PK), customer_id (FK), account_id (FK), loan_type, amount, interest_rate, start_date, end_date, status	Loan master; end date after start date
LOAN_SCHEDULE	schedule_id (PK), loan_id (FK), due_date, due_amount, status	One EMI per row; unique per loan and due date
LOAN_REPAYMENT	repayment_id (PK), transaction_id (FK), loan_id (FK)	Cash payment made against a loan
REPAYMENT	repayment_id (PK), schedule_id (FK), transaction_id (FK), amount, status	Allocation of a payment to an instalment
BRANCH 1 : N ACCOUNT	branch_id in ACCOUNT	Total participation of ACCOUNT; a branch may have no account
ACCOUNT_TYPE 1 : N ACCOUNT	type_id in ACCOUNT	Every account has exactly one type
CUSTOMER M : N ACCOUNT	through ACCOUNT_HOLDER	A joint account has several holders; a customer may hold several accounts
ACCOUNT 1 : N TRANSACTION	account_id in TRANSACTION	Ledger of the account; every posting belongs to one account
TRANSACTION 1 : 0..1 TRANSFER (two roles)	debit_ and credit_transaction_id	A transfer owns exactly two postings, one per role
CUSTOMER 1 : N LOAN, ACCOUNT 1 : N LOAN	customer_id, account_id in LOAN	A loan has one borrower and one disbursement account
LOAN 1 : N LOAN_SCHEDULE	loan_id in LOAN_SCHEDULE	Instalments of the loan
LOAN_SCHEDULE 1 : N REPAYMENT	schedule_id in REPAYMENT	A payment can be allocated to several instalments

CHAPTER 2

LOGICAL DATABASE DESIGN

2.1 Relational Schema

Every entity in Figure 1 became one relation, giving the 14 tables in Table 5. Relationships became foreign keys, and the many-to-many relationship between customers and accounts became the ACCOUNT_HOLDER table. The relational schema is also written in compact notation below the table, with primary key columns tagged PK and foreign key columns tagged FK.

Table 5. Relational Schema

S.No.	Table Name	Purpose
1	branch	Bank branches and their managers
2	account_type	Account products and their minimum balance rules
3	customer	Customer profile and identity data
4	account	Bank accounts with status and trigger-maintained balance
5	account_holder	Links customers to accounts (primary and joint holders)
6	transaction	Ledger of every posting with balance snapshot
7	transfer	Pairs a debit posting with a credit posting
8	service_charge	Marks a posting as a service charge and its type
9	beneficiary	Saved payees of a customer
10	statement	Log of generated account statements
11	loan	Loan master record
12	loan_schedule	EMI schedule, one row per instalment
13	loan_repayment	Marks a posting as a loan payment
14	repayment	Allocates a payment to a schedule instalment

Relational schema in compact notation:

```
branch(branch_id [PK], branch_name, address, city, phone, branch_manager)
account_type(type_id [PK], type_name, description, min_balance,
is_interest_bearing)
customer(customer_id [PK], full_name, date_of_birth, email, phone, address,
id_proof)
account(account_id [PK], account_number, type_id [FK], branch_id [FK], status,
open_date, balance)
account_holder(account_id [PK FK], customer_id [PK FK], holder_type, since_date)
transaction(transaction_id [PK], account_id [FK], transaction_type, amount,
transaction_date, description, balance_after)
transfer(transfer_id [PK], debit_transaction_id [FK], credit_transaction_id [FK],
remarks)
service_charge(charge_id [PK], transaction_id [FK], charge_type)
beneficiary(beneficiary_id [PK], customer_id [FK], beneficiary_name,
account_number, bank_name, ifsc_code, type, created_at)
statement(statement_id [PK], account_id [FK], from_date, to_date, generated_date)
loan(loan_id [PK], customer_id [FK], account_id [FK], loan_type, amount,
interest_rate, start_date, end_date, status)
loan_schedule(schedule_id [PK], loan_id [FK], due_date, due_amount, status)
loan_repayment(repayment_id [PK], transaction_id [FK], loan_id [FK])
repayment(repayment_id [PK], schedule_id [FK], transaction_id [FK], amount, status)
```

2.2 Keys and Integrity Constraints

Table 6 lists, for every table, the keys and integrity constraints declared in the DDL. Foreign keys use the default RESTRICT action, so a parent row cannot be deleted while children exist; accounts and loans are therefore closed by changing their status rather than deleted. Not-null and default constraints are shown in aggregate for each table.

Table 6. Keys and Integrity Constraints

Table Name	Key / Constraint	Attribute(s)	Business Rule Enforced
branch	Primary Key	branch_id	Unique identity of each row
	Unique	branch_name	No duplicate values
	Not Null	All columns	Mandatory data must be supplied
account_type	Primary Key	type_id	Unique identity of each row
	Unique	type_name	No duplicate values
	Check	min_balance >= 0	Minimum balance cannot be negative
	Not Null	All columns except description	Mandatory data must be supplied
	Default	min_balance = 0.00; is_interest_bearing = 1	Sensible value when none is given
customer	Primary Key	customer_id	Unique identity of each row
	Unique	email	No duplicate values
	Unique	id_proof	No duplicate values
	Check	email LIKE '%_@_%._%'	E-mail must look like name@domain.tld
	Not Null	All columns	Mandatory data must be supplied
account	Primary Key	account_id	Unique identity of each row
	Foreign Key	branch_id -> branch	Referential integrity (RESTRICT): no orphan rows
	Foreign Key	type_id -> account_type	Referential integrity (RESTRICT): no orphan rows
	Unique	account_number	No duplicate values
	Check	balance >= 0	Stored balance never negative
	Not Null	All columns	Mandatory data must be supplied
	Default	status = ACTIVE; balance = 0.00	Sensible value when none is given
account_holder	Primary Key	account_id, customer_id	Unique identity of each row (composite)
	Foreign Key	account_id -> account	Referential integrity (RESTRICT): no orphan rows
	Foreign Key	customer_id -> customer	Referential integrity (RESTRICT): no orphan rows
	Not Null	All columns	Mandatory data must be supplied
	Default	holder_type = PRIMARY	Sensible value when none is given
transaction	Primary Key	transaction_id	Unique identity of each row
	Foreign Key	account_id -> account	Referential integrity (RESTRICT): no orphan rows
	Check	amount > 0	Every posting amount is positive
	Not Null	All columns except description, balance_after	Mandatory data must be supplied
	Default	transaction_date = current_timestamp()	Sensible value when none is given
transfer	Primary Key	transfer_id	Unique identity of each row
	Foreign Key	credit_transaction_id -> transaction	Referential integrity (RESTRICT): no orphan rows
	Foreign Key	debit_transaction_id -> transaction	Referential integrity (RESTRICT): no orphan rows
	Unique	credit_transaction_id	No duplicate values
	Unique	debit_transaction_id	No duplicate values

Table Name	Key / Constraint	Attribute(s)	Business Rule Enforced
	Check	debit_transaction_id <> credit_transaction_id	Debit and credit legs are different postings
	Not Null	All columns except remarks	Mandatory data must be supplied
service_charge	Primary Key	charge_id	Unique identity of each row
	Foreign Key	transaction_id -> transaction	Referential integrity (RESTRICT): no orphan rows
	Unique	transaction_id	No duplicate values
	Not Null	All columns	Mandatory data must be supplied
beneficiary	Primary Key	beneficiary_id	Unique identity of each row
	Foreign Key	customer_id -> customer	Referential integrity (RESTRICT): no orphan rows
	Unique	customer_id, account_number, ifsc_code	No duplicate values
	Not Null	All columns	Mandatory data must be supplied
	Default	type = INTERNAL; created_at = current_timestamp()	Sensible value when none is given
statement	Primary Key	statement_id	Unique identity of each row
	Foreign Key	account_id -> account	Referential integrity (RESTRICT): no orphan rows
	Check	to_date >= from_date	Statement end date is not before start date
	Not Null	All columns	Mandatory data must be supplied
	Default	generated_date = current_timestamp()	Sensible value when none is given
loan	Primary Key	loan_id	Unique identity of each row
	Foreign Key	account_id -> account	Referential integrity (RESTRICT): no orphan rows
	Foreign Key	customer_id -> customer	Referential integrity (RESTRICT): no orphan rows
	Check	amount > 0	Loan amount is positive
	Check	interest_rate > 0	Interest rate is positive
	Check	end_date > start_date	Loan ends after it starts
	Not Null	All columns	Mandatory data must be supplied
	Default	status = ACTIVE	Sensible value when none is given
loan_schedule	Primary Key	schedule_id	Unique identity of each row
	Foreign Key	loan_id -> loan	Referential integrity (RESTRICT): no orphan rows
	Unique	loan_id, due_date	No duplicate values
	Check	due_amount > 0	EMI amount is positive
	Not Null	All columns	Mandatory data must be supplied
	Default	status = PENDING	Sensible value when none is given
loan_repayment	Primary Key	repayment_id	Unique identity of each row
	Foreign Key	loan_id -> loan	Referential integrity (RESTRICT): no orphan rows
	Foreign Key	transaction_id -> transaction	Referential integrity (RESTRICT): no orphan rows
	Unique	transaction_id	No duplicate values
	Not Null	All columns	Mandatory data must be supplied
repayment	Primary Key	repayment_id	Unique identity of each row
	Foreign Key	schedule_id -> loan_schedule	Referential integrity (RESTRICT): no orphan rows
	Foreign Key	transaction_id -> transaction	Referential integrity (RESTRICT): no orphan rows
	Check	amount > 0	Allocated amount is positive
	Not Null	All columns	Mandatory data must be supplied
	Default	status = APPLIED	Sensible value when none is given

2.3 Functional Dependencies

The functional dependencies below were used to justify the decomposition [3]. In each relation the left-hand side is a key (primary or candidate), so no non-key attribute depends on another non-key attribute.

BRANCH: branch_id → branch_name, address, city, phone, branch_manager

ACCOUNT_TYPE: type_id → type_name, description, min_balance, is_interest_bearing

CUSTOMER: customer_id → full_name, date_of_birth, email, phone, address, id_proof; email → customer_id; id_proof → customer_id

ACCOUNT: account_id → account_number, type_id, branch_id, status, open_date, balance; account_number → account_id

ACCOUNT_HOLDER: (account_id, customer_id) → holder_type, since_date

TRANSACTION: transaction_id → account_id, transaction_type, amount, transaction_date, description, balance_after

TRANSFER: transfer_id → debit_transaction_id, credit_transaction_id, remarks

SERVICE_CHARGE: charge_id → transaction_id, charge_type

BENEFICIARY: beneficiary_id → customer_id, beneficiary_name, account_number, bank_name, ifsc_code, type, created_at; (customer_id, account_number, ifsc_code) → beneficiary_id

STATEMENT: statement_id → account_id, from_date, to_date, generated_date

LOAN: loan_id → customer_id, account_id, loan_type, amount, interest_rate, start_date, end_date, status

LOAN_SCHEDULE: schedule_id → loan_id, due_date, due_amount, status; (loan_id, due_date) → schedule_id

LOAN_REPAYMENT: repayment_id → transaction_id, loan_id

REPAYMENT: repayment_id → schedule_id, transaction_id, amount, status

2.4 Normalization up to Third Normal Form (3NF)

Normalisation started from one bank sheet (UNF) that repeated customer, branch, account, joint holders, many transactions and loan instalments in a single record. First normal form [3] was reached by making every value atomic, moving the repeating groups (holders, transactions, instalments) into their own rows and giving every table a primary key. Second normal form removed partial dependencies: branch and account-type data were split out of ACCOUNT, and ACCOUNT_HOLDER was created because holder details depend on the pair (account, customer). Third normal form removed transitive dependencies: TRANSFER, SERVICE_CHARGE and LOAN_REPAYMENT became subtypes of TRANSACTION, and REPAYMENT allocates a payment to a LOAN_SCHEDULE instalment.

An example of an anomaly that disappeared: before decomposition each account row carried the branch name, city and manager, so changing a manager meant editing every account of

that branch; now ACCOUNT stores only branch_id and BRANCH holds the details once. Because every determinant in Table 7 is a candidate key, each relation also satisfies Boyce-Codd normal form [4]. Two values are deliberately redundant: account.balance and transaction.balance_after are derived from the ledger and are written only by triggers, which keeps balance look-ups fast; an audit query confirms they never disagree with the ledger.

Table 7 records the normal form reached by every relation.

Table 7. Normalization Status up to 3NF

Relation Name	1NF	2NF	3NF	BCNF	Remarks / Decomposition
branch	Yes	Yes	Yes	Yes	Atomic; single-attribute key; no transitive dependency.
account_type	Yes	Yes	Yes	Yes	Product rules stored once, not per account.
customer	Yes	Yes	Yes	Yes	email and id_proof are candidate keys.
account	Yes	Yes	Yes	Yes	Branch and type split out; balance is controlled redundancy.
account_holder	Yes	Yes	Yes	Yes	Created to resolve customer M : N account; composite key.
transaction	Yes	Yes	Yes	Yes	balance_after is a trigger-set audit snapshot.
transfer	Yes	Yes	Yes	Yes	Subtype of TRANSACTION; two UNIQUE legs.
service_charge	Yes	Yes	Yes	Yes	Subtype of TRANSACTION; charge_type depends on the key only.
beneficiary	Yes	Yes	Yes	Yes	Candidate key (customer_id, account_number, ifsc_code).
statement	Yes	Yes	Yes	Yes	Request log; no derived values stored.
loan	Yes	Yes	Yes	Yes	Customer and account referenced by key only.
loan_schedule	Yes	Yes	Yes	Yes	Candidate key (loan_id, due_date); one row per EMI.
loan_repayment	Yes	Yes	Yes	Yes	Subtype of TRANSACTION for loan payments.
repayment	Yes	Yes	Yes	Yes	Allocation row splits a payment across instalments.

2.5 Data Dictionary

Table 8 is the data dictionary of all 14 tables, generated from the live schema. Types are those of MySQL; PK, FK, UQ, NN and DF mean primary key, foreign key, unique, not null and default.

Table 8. Data Dictionary

Table	Attribute	Data Type	Key / Constraint	Description
branch	branch_id	INT	PK, AI	Branch identifier
branch	branch_name	VARCHAR(80)	UQ, NN	Branch name
branch	address	VARCHAR(150)	NN	Street address
branch	city	VARCHAR(50)	NN	City
branch	phone	VARCHAR(15)	NN	Contact number
branch	branch_manager	VARCHAR(80)	NN	Name of the manager
account_type	type_id	INT	PK, AI	Account type identifier
account_type	type_name	VARCHAR(40)	UQ, NN	Savings, Current, Salary, Fixed Deposit
account_type	description	VARCHAR(150)		Short description of the product
account_type	min_balance	DECIMAL(12,2)	NN, DF 0.00	Minimum balance that must remain after a debit
account_type	is_interest_bearing	TINYINT(1)	NN, DF 1	1 if the product earns interest
customer	customer_id	INT	PK, AI	Customer identifier
customer	full_name	VARCHAR(100)	NN	Customer name
customer	date_of_birth	DATE	NN	Date of birth
customer	email	VARCHAR(120)	UQ, NN	Unique e-mail address

Table	Attribute	Data Type	Key / Constraint	Description
customer	phone	VARCHAR(15)	NN	Mobile number
customer	address	VARCHAR(150)	NN	Residential address
customer	id_proof	VARCHAR(30)	UQ, NN	Unique ID proof number (PAN, Aadhaar, passport)
account	account_id	INT	PK, AI	Account identifier
account	account_number	VARCHAR(16)	UQ, NN	Unique 14-digit account number
account	type_id	INT	FK, NN	Account type of this account
account	branch_id	INT	FK, NN	Branch holding the account
account	status	ENUM	NN, DF ACTIVE	ACTIVE, DORMANT, FROZEN or CLOSED
account	open_date	DATE	NN	Date the account was opened
account	balance	DECIMAL(15,2)	NN, DF 0.00	Current balance, maintained only by trigger
account_holder	account_id	INT	PK, FK	Account held
account_holder	customer_id	INT	PK, FK	Customer who holds it
account_holder	holder_type	ENUM	NN, DF PRIMARY	PRIMARY or JOINT
account_holder	since_date	DATE	NN	Date the holder was added
transaction	transaction_id	BIGINT	PK, AI	Posting identifier
transaction	account_id	INT	FK, NN	Account that is posted to
transaction	transaction_type	ENUM	NN	Deposit, withdrawal, transfer leg, charge, loan item
transaction	amount	DECIMAL(15,2)	NN	Posting amount, always positive
transaction	transaction_date	DATETIME	NN, DF NOW	Date and time of posting
transaction	description	VARCHAR(150)		Free-text narration
transaction	balance_after	DECIMAL(15,2)		Account balance after this posting (trigger-set)
transfer	transfer_id	INT	PK, AI	Transfer identifier
transfer	debit_transaction_id	BIGINT	FK, UQ, NN	Posting that debited the source
transfer	credit_transaction_id	BIGINT	FK, UQ, NN	Posting that credited the target
transfer	remarks	VARCHAR(150)		Reference or remark
service_charge	charge_id	INT	PK, AI	Charge identifier
service_charge	transaction_id	BIGINT	FK, UQ, NN	Posting that carried the charge
service_charge	charge_type	ENUM	NN	SMS_ALERT, ATM_FEE, MIN_BALANCE_PENALTY, CHEQUE_BOOK, ANNUAL_MAINTENANCE
beneficiary	beneficiary_id	INT	PK, AI	Beneficiary identifier
beneficiary	customer_id	INT	FK, NN	Customer who saved the payee
beneficiary	beneficiary_name	VARCHAR(100)	NN	Payee name
beneficiary	account_number	VARCHAR(20)	NN	Payee account number
beneficiary	bank_name	VARCHAR(80)	NN	Payee bank
beneficiary	ifsc_code	VARCHAR	NN	Payee branch IFSC
beneficiary	type	ENUM	NN, DF INTERNAL	INTERNAL (this bank) or EXTERNAL
beneficiary	created_at	DATETIME	NN, DF NOW	When the payee was saved
statement	statement_id	INT	PK, AI	Statement request identifier
statement	account_id	INT	FK, NN	Account of the statement
statement	from_date	DATE	NN	Period start
statement	to_date	DATE	NN	Period end
statement	generated_date	DATETIME	NN, DF NOW	When it was generated
loan	loan_id	INT	PK, AI	Loan identifier
loan	customer_id	INT	FK, NN	Borrower
loan	account_id	INT	FK, NN	Disbursement and repayment account
loan	loan_type	ENUM	NN	PERSONAL, VEHICLE, EDUCATION or HOME
loan	amount	DECIMAL(15,2)	NN	Principal
loan	interest_rate	DECIMAL(5,2)	NN	Annual interest rate in percent
loan	start_date	DATE	NN	Disbursement date
loan	end_date	DATE	NN	Date of the last instalment

Table	Attribute	Data Type	Key / Constraint	Description
loan	status	ENUM	NN, DF ACTIVE	ACTIVE, CLOSED or DEFAULTED
loan_schedule	schedule_id	INT	PK, AI	Instalment identifier
loan_schedule	loan_id	INT	FK, NN	Loan of the instalment
loan_schedule	due_date	DATE	NN	Due date
loan_schedule	due_amount	DECIMAL(15,2)	NN	EMI amount
loan_schedule	status	ENUM	NN, DF PENDING	PENDING, PAID or OVERDUE
loan_repayment	repayment_id	INT	PK, AI	Loan payment identifier
loan_repayment	transaction_id	BIGINT	FK, UQ, NN	Posting of the payment
loan_repayment	loan_id	INT	FK, NN	Loan paid
repayment	repayment_id	INT	PK, AI	Allocation identifier
repayment	schedule_id	INT	FK, NN	Instalment paid
repayment	transaction_id	BIGINT	FK, NN	Payment posting
repayment	amount	DECIMAL(15,2)	NN	Amount allocated to the instalment
repayment	status	ENUM	NN, DF APPLIED	APPLIED or REVERSED

2.6 Implementation of Logical Design: DDL and Sample Data

The design was implemented in MySQL 8 [5] using the InnoDB engine. Table 9 describes the environment. The script was also verified on MariaDB 10.11 during development. Representative DDL is shown below; the complete script is in Appendix A.

Table 9. Database Implementation Environment

Item	Details
DBMS	MySQL 8.x (InnoDB); script also verified on MariaDB 10.11
DBMS Version	MySQL 8.0 or later (CHECK constraints are enforced from 8.0.16)
Number of Tables	14 base tables, 7 views, 3 triggers, 2 stored procedures, 5 secondary indexes (plus the application table staff_user)
Sample Data	4 branches, 10 customers, 11 accounts, 54 ledger postings, 8 transfers, 3 loans, 27 instalments, 7 beneficiaries, 6 statements, 4 service charges

DDL of the ACCOUNT and TRANSACTION tables:

```
CREATE TABLE account (
    account_id      INT AUTO_INCREMENT PRIMARY KEY,
    account_number  VARCHAR(16)  NOT NULL UNIQUE,
    type_id         INT          NOT NULL,
    branch_id       INT          NOT NULL,
    status          ENUM('ACTIVE','DORMANT','FROZEN','CLOSED')
                    NOT NULL DEFAULT 'ACTIVE',
    open_date       DATE         NOT NULL,
    balance         DECIMAL(15,2) NOT NULL DEFAULT 0.00,
    CONSTRAINT ck_account_balance CHECK (balance >= 0),
    CONSTRAINT fk_account_type   FOREIGN KEY (type_id)
        REFERENCES account_type(type_id),
    CONSTRAINT fk_account_branch FOREIGN KEY (branch_id)
        REFERENCES branch(branch_id)
) ENGINE=InnoDB;
```

```
CREATE TABLE `transaction` (
    transaction_id    BIGINT AUTO_INCREMENT PRIMARY KEY,
    account_id        INT NOT NULL,
    transaction_type  ENUM('DEPOSIT','WITHDRAWAL','TRANSFER_DEBIT',
        'TRANSFER_CREDIT','SERVICE_CHARGE','LOAN_DISBURSEMENT',
        'LOAN_REPAYMENT') NOT NULL,
    amount            DECIMAL(15,2) NOT NULL,
    transaction_date  DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    description       VARCHAR(150),
    balance_after     DECIMAL(15,2) NULL,
    CONSTRAINT ck_txn_amount_positive CHECK (amount > 0),
    CONSTRAINT fk_txn_account FOREIGN KEY (account_id)
        REFERENCES account(account_id)
) ENGINE=InnoDB;
```

The BEFORE INSERT trigger on TRANSACTION is the heart of the rule engine. It locks the account row, rejects bad postings with SIGNAL SQLSTATE '45000' [5], and writes the balance snapshot; an AFTER INSERT trigger copies the snapshot into account.balance.

```
CREATE TRIGGER trg_txn_before_insert BEFORE INSERT ON `transaction`
FOR EACH ROW BEGIN
  ... SELECT a.status, a.balance, t.min_balance
        INTO v_status, v_bal, v_min
        FROM account a JOIN account_type t ON t.type_id = a.type_id
       WHERE a.account_id = NEW.account_id FOR UPDATE;
  IF NEW.amount IS NULL OR NEW.amount <= 0 THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Transaction amount must be positive';
  END IF;
  IF v_status <> 'ACTIVE' THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Account is not ACTIVE - transaction rejected';
  END IF;
  ... debit: IF v_bal - NEW.amount < v_min THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Insufficient balance (minimum balance rule)';
  SET NEW.balance_after = v_new;
END;
```

The database is loaded with realistic sample data (record counts in Table 9): four branches in Telangana, ten customers, eleven accounts including one joint account, one FROZEN and one CLOSED account, 54 ledger postings, 8 transfers, three loans with 27 instalments (8 paid, 2 overdue) and 7 saved beneficiaries. Deposits across all accounts total INR 15,73,925. Figure 2 shows the 14 base tables in MySQL.

```
mysql> SELECT account_number, type_name, branch_name, status, balance
       FROM v_account_balances ORDER BY balance DESC LIMIT 5;
+----------------+---------------+-------------------+--------+-----------+
| account_number | type_name     | branch_name       | status | balance   |
+----------------+---------------+-------------------+--------+-----------+
| 50010000001003 | Fixed Deposit | Hanamkonda Main   | ACTIVE | 500000.00 |
| 50010000003001 | Current       | Hyderabad Banjara | ACTIVE | 445438.88 |
| 50010000004002 | Savings       | Karimnagar Town   | ACTIVE | 150624.28 |
| 50010000001001 | Savings       | Hanamkonda Main   | ACTIVE | 110157.22 |
| 50010000003002 | Savings       | Hyderabad Banjara | ACTIVE |  98000.00 |
+----------------+---------------+-------------------+--------+-----------+
```

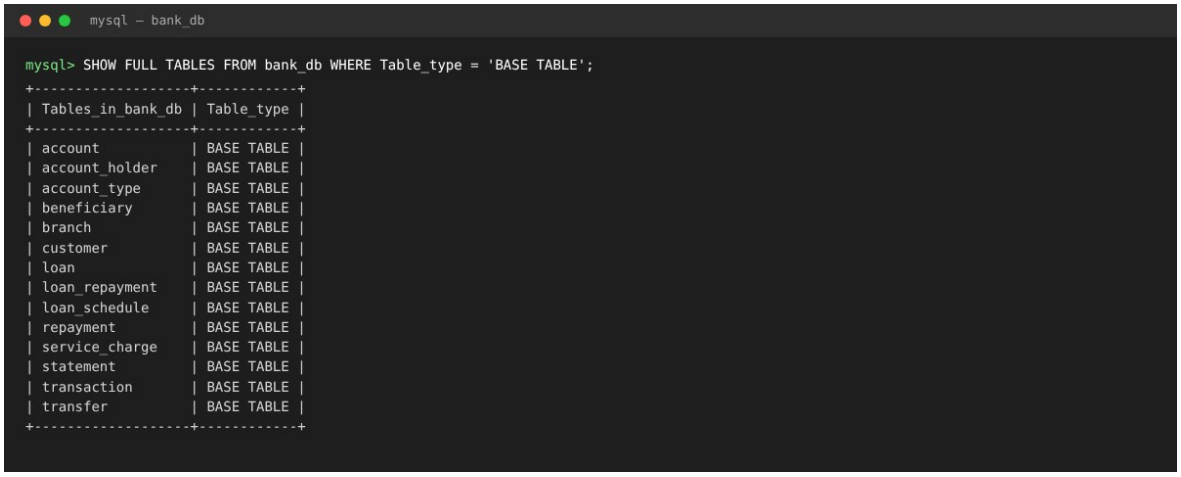


Figure 2. The 14 base tables in the bank_db database (MySQL)

CHAPTER 3

APPLICATION DEVELOPMENT USING LOGICAL DATABASE DESIGN

3.1 Application Overview

The application is a web system named 'Woxsen Co-operative Bank' for bank staff. Tellers, loan officers and branch managers sign in with their own role and use nine screens: Login, Dashboard, Customers, Accounts, Transactions, Transfers, Statements, Loans and Rule Tester. The application contains no banking data of its own. Every screen reads from and writes to the MySQL database through a REST API; balances, statements and loan figures are always taken from the tables and views of Chapter 2, and every monetary rule is enforced by the database triggers and procedures, not by the browser. The Rule Tester screen deliberately tries seven illegal operations inside a transaction that is always rolled back, which lets a user watch the database refuse each one.

3.2 Development Environment and Database Connectivity

Table 10 lists the development environment. Connectivity is centralised in one module, db.js, which creates a pool of ten connections and exposes two helpers: query() runs a prepared statement, and withTransaction() runs a block on one connection and commits on success or rolls back on any error. Dates and DECIMAL values are returned as strings, which avoids time-zone shifts and floating-point errors in money. Besides the 14 banking tables, the application creates one extra table, staff_user, at start-up to hold staff logins (username, role and a bcrypt hash).

Table 10. Application Development Environment

Item	Details
Programming Language	JavaScript: Node.js 18+ on the server, vanilla JavaScript (ES modules) in the browser
DBMS	MySQL 8 (InnoDB)
IDE / Framework / GUI Toolkit	Express 5 web framework [6]; HTML5, CSS3 and a hand-written single-page front end; Visual Studio Code
Database Connector / Driver	mysql2 (promise API, connection pool of 10, prepared statements); JSON Web Tokens [7] in an HTTP-only cookie; bcryptjs for password hashes
Operating Environment	macOS and Linux; the browser needs no plug-ins; configuration through a .env file
Repository Link	https://github.com/srilakshmireddy-byte/bank-dbms-pbl

3.3 Application Modules and Database Mapping

Table 11 maps each application module to the tables and views it uses. Every module goes through the database; none keeps data in the browser.

Table 11. Application Module-to-Database Mapping

Module / Screen	Purpose	Tables Used	Main Operations
Login	Authenticate staff and issue a signed session	staff_user	Read, verify bcrypt hash
Dashboard	Totals, monthly deposits and withdrawals, latest postings, branch cards	v_branch_summary, v_account_balances, v_loan_outstanding, transaction	Read, aggregate
Customers	Search, register and update customers	customer, account_holder, account	Create, read, update
Accounts	List and search accounts, open an account with holders, change status	account, account_holder, account_type, branch, customer	Create, read, update status
Transactions	Post deposits and withdrawals; audit log	transaction, account, v_transaction_audit	Create, read
Transfers	Atomic transfer; manage beneficiaries	transfer, transaction, beneficiary, v_transfers	Create, read, delete beneficiary (CALL sp_transfer)
Statements	Statement for any period with running balance	statement, transaction, account	Create log row, read
Loans	Create loan with schedule; collect repayments	loan, loan_schedule, loan_repayment, repayment, v_loan_outstanding, v_repayment_history	Create, read, update (CALL sp_loan_repayment)
Rule Tester	Run seven illegal operations inside a rolled-back transaction	transaction, transfer, account	Attempted writes, all rolled back

3.4 CRUD Operations

Table 12 shows how the four CRUD operations are realised. Financial records are never physically deleted.

Table 12. CRUD Operations

Operation	Application Feature / Screen	Table(s) Affected	Evidence
Create	Register customer; open account; post deposit; transfer money; create loan; generate statement; save beneficiary	customer, account, account_holder, transaction, transfer, loan, loan_schedule, statement, beneficiary	Figures 6 and 8; TC-04, TC-09
Read / Search	Customer and account search; dashboard; statements; loan list and schedule; audit log	customer, account, transaction, loan, loan_schedule and the 7 views	Figures 4, 5 and 10; TC-03, TC-14
Update	Edit customer details; freeze or close an account; repay an instalment (marks it PAID and may close the loan)	customer, account, loan_schedule, loan, repayment	Figure 9; TC-12
Delete / Controlled Removal	Money records are never deleted: accounts are closed by status and foreign keys are RESTRICT; only a saved beneficiary can be removed	account (status), beneficiary	TC-08; Table 6

3.5 Business Rules, Validation and Error Handling

All SQL in the application uses parameterised (prepared) statements, so user input is never concatenated into a query; this is the primary defence against SQL injection [8]. The

application checks only the shape of the input (a number, a date, an identifier). Business rules are left to the database on purpose, so that the same rule applies to every client. When the database rejects an operation, the SQLSTATE 45000 message is returned as HTTP 422 and shown to the user verbatim. Multi-step actions such as a transfer, a loan creation or a repayment run as one transaction, and any failure rolls everything back. Table 13 lists the rules.

Table 13. Business Rules, Validation and Error Handling

Rule / Validation	How Implemented	Evidence / Message
Amount must be positive	BEFORE INSERT trigger on transaction (plus CHECK ck_txn_amount_positive)	'Transaction amount must be positive' (HTTP 422)
No overdrawing / minimum balance	Trigger compares balance minus amount with the account type's min_balance, row locked with FOR UPDATE	'Insufficient balance (minimum balance rule)'
Only ACTIVE accounts accept postings	Trigger checks account.status	'Account is not ACTIVE - transaction rejected'
Transfer needs two different accounts	sp_transfer raises SIGNAL before any posting	'Cannot transfer to the same account'
Transfer legs must balance	trg_transfer_before_insert and stored procedure in one transaction with EXIT HANDLER ROLLBACK	'Unbalanced transfer: debit must equal credit'
Duplicate customer, account number	UNIQUE constraints; error code 1062 mapped to HTTP 409	'Duplicate entry ...' shown to the user
Malformed input (amount, date, id)	Shape validation in lib/validate.js before the query	'Amount must be a number with at most 2 decimals' (HTTP 400)
Statement period valid	CHECK ck_statement_period on statement	Constraint violation reported as HTTP 422
Instalment cannot be paid twice	Application guard before sp_loan_repayment	'This instalment is already fully paid' (HTTP 409)
Role-based access	Permission table in lib/permissions.js checked on every route	'Your role (teller) is not allowed to do this' (HTTP 403)

3.6 Reports and Database Operations

Table 14 lists the reports and the SQL concepts behind them. The statement, dashboard and loan reports are computed by the database, not in the browser.

Table 14. Reports and Database Operations

Report / Operation	SQL / Database Concept Used	Application Output
Dashboard totals	Aggregation (SUM, COUNT) over views v_branch_summary and v_loan_outstanding	Cards: deposits held, active accounts, loans outstanding, overdue instalments
Account statement	JOIN of transaction and account; running balance rebuilt in date order; opening and closing balance by subquery	Statement of Account (Figure 10)
Loan outstanding	View with SUM of repayments grouped by loan; total payable less repaid	Loans table with progress bars
Overdue instalments	WHERE status = 'OVERDUE' on index idx_schedule_status; overdue marking at start-up and hourly	Overdue card and loan filter
Branch summary	GROUP BY branch with COUNT, SUM, AVG over accounts	Branch cards on the dashboard
Monthly deposits and withdrawals	Conditional aggregation: SUM(CASE WHEN type = ...) GROUP BY month	Bar chart on the dashboard

Report / Operation	SQL / Database Concept Used	Application Output
Transaction audit	View joining ledger, account, holders and branch; filters and paging	Audit log on the Transactions page (Manager)
Ledger integrity check	Correlated subquery: balance versus last ledger balance_after	0 rows returned when consistent (Section 3.8)

3.7 Application Screens

Figures 3 to 11 are screenshots of the running application connected to the MySQL database. Figure 3 is the staff login. Figure 4 is the manager's dashboard. Figure 5 searches accounts. Figure 6 shows data entry: a cash deposit posted to an account, which the database confirms as a new transaction. Figure 7 shows the database refusing a withdrawal larger than the balance. Figure 8 is a transfer between two accounts, a project-specific transaction. Figure 9 shows a loan after an instalment is repaid (an update). Figure 10 is the statement report, and Figure 11 is the Rule Tester.

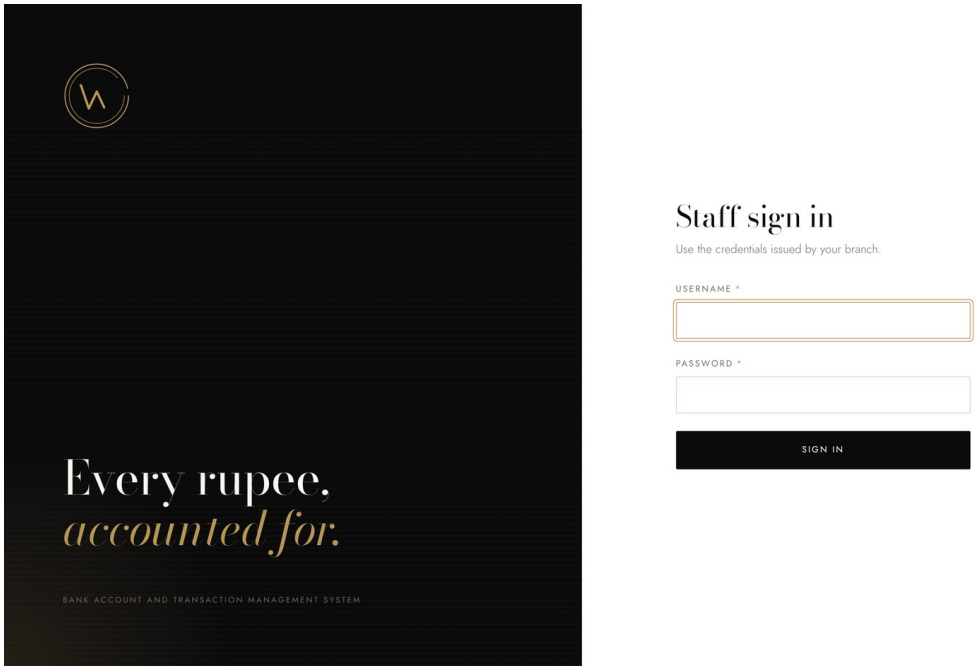


Figure 3. Staff Sign-in Screen

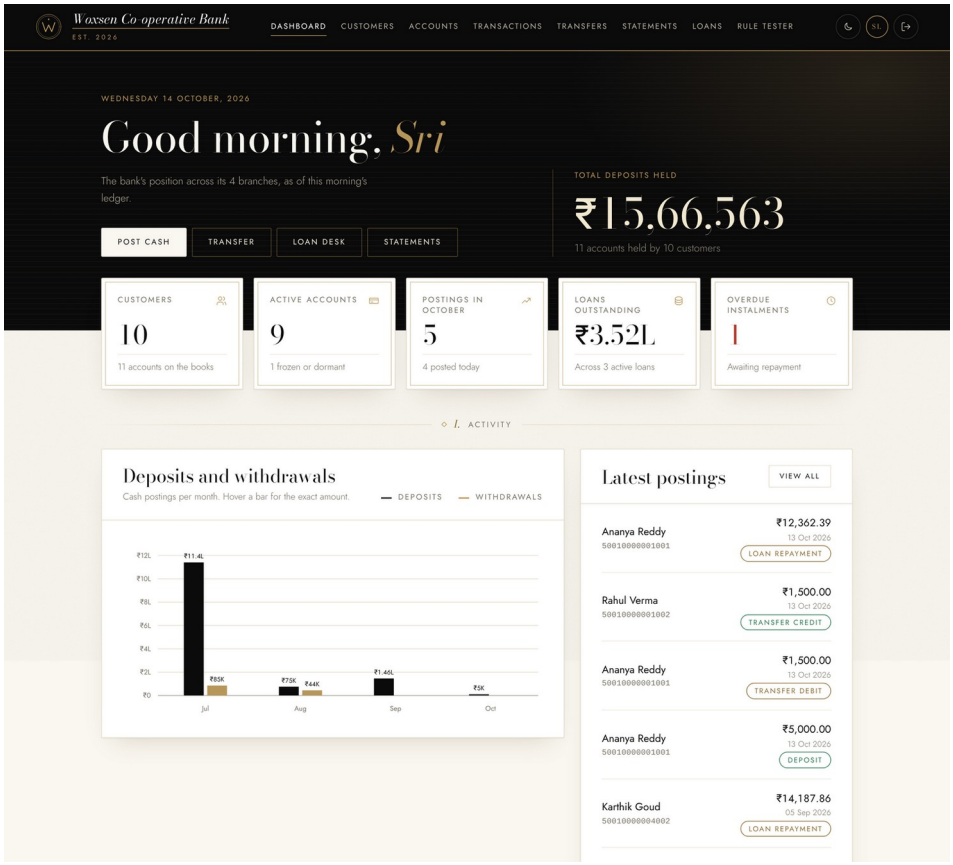


Figure 4. Manager Dashboard with Live Totals and Branch Summary

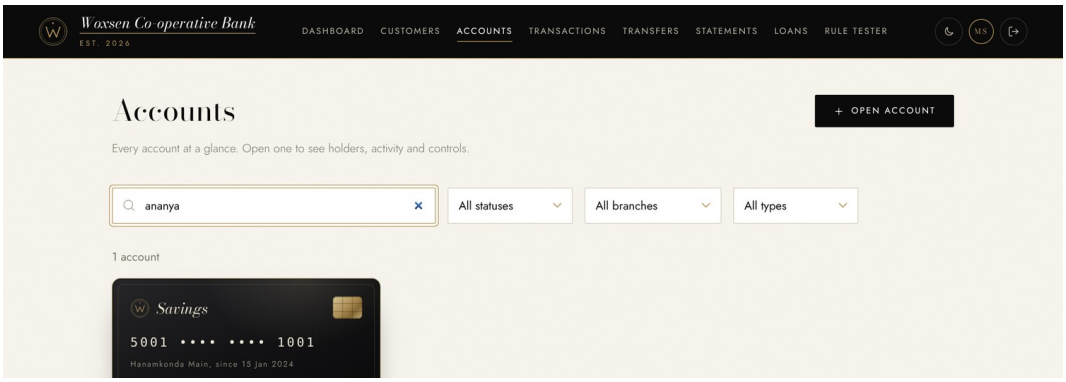


Figure 5. Account Search and Display

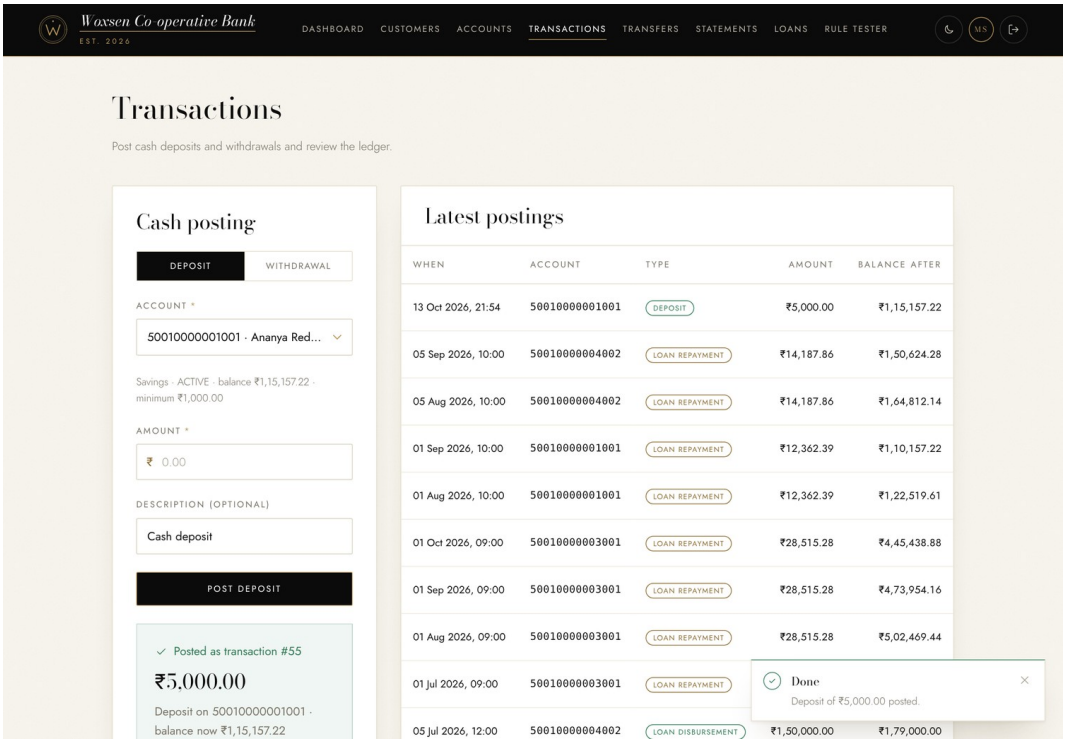


Figure 6. Cash Deposit Posted to an Account (Data Entry)

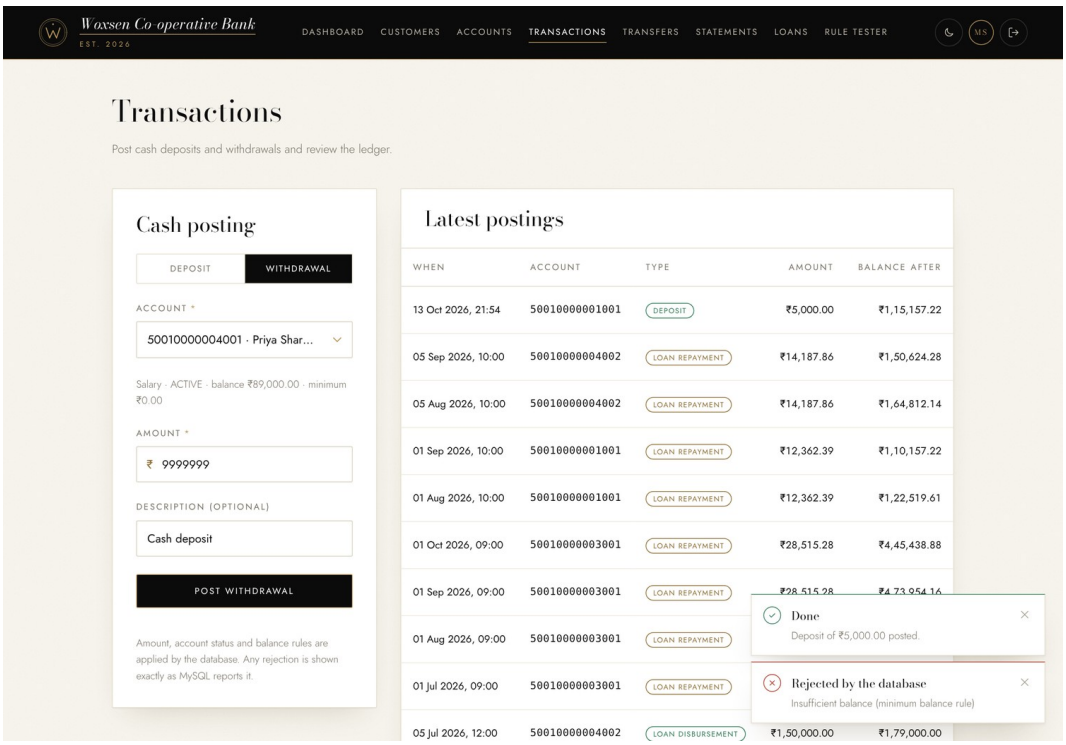


Figure 7. Withdrawal Beyond the Balance Rejected by the Database

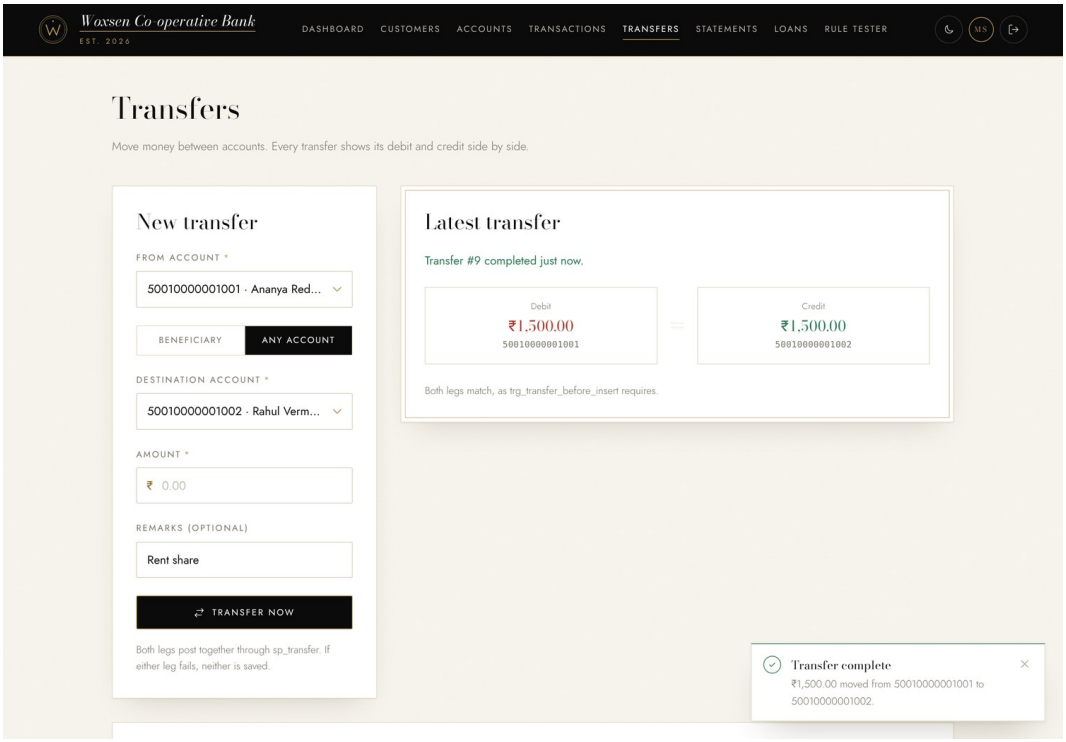


Figure 8. Atomic Transfer Between Two Accounts

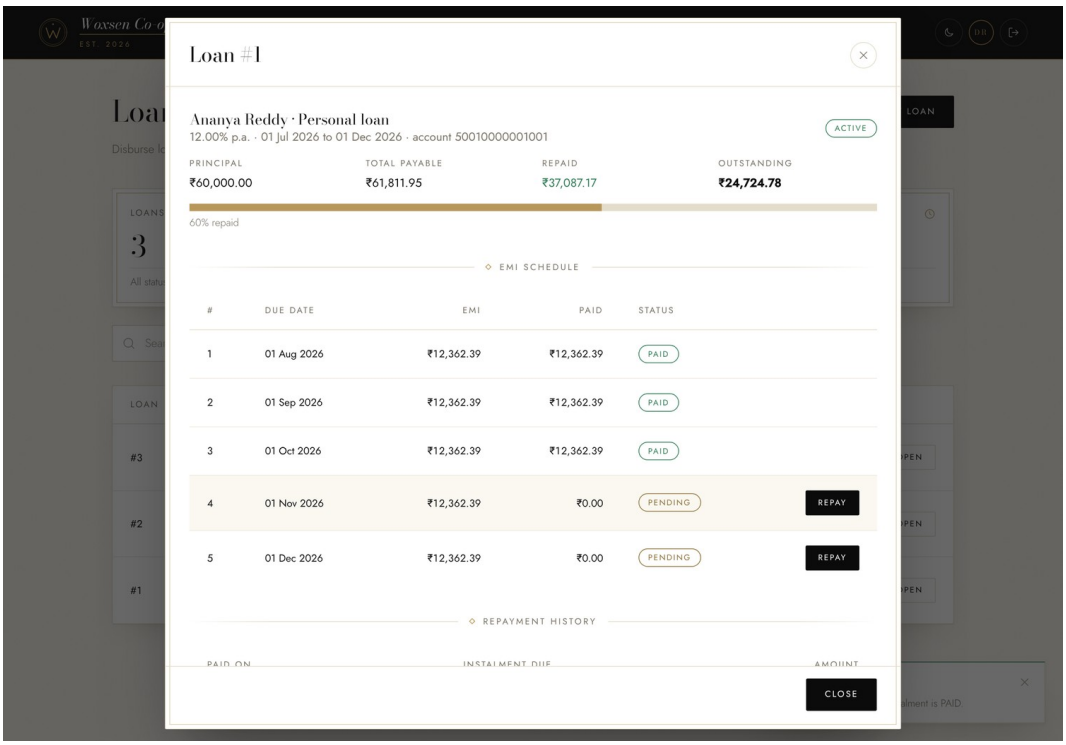
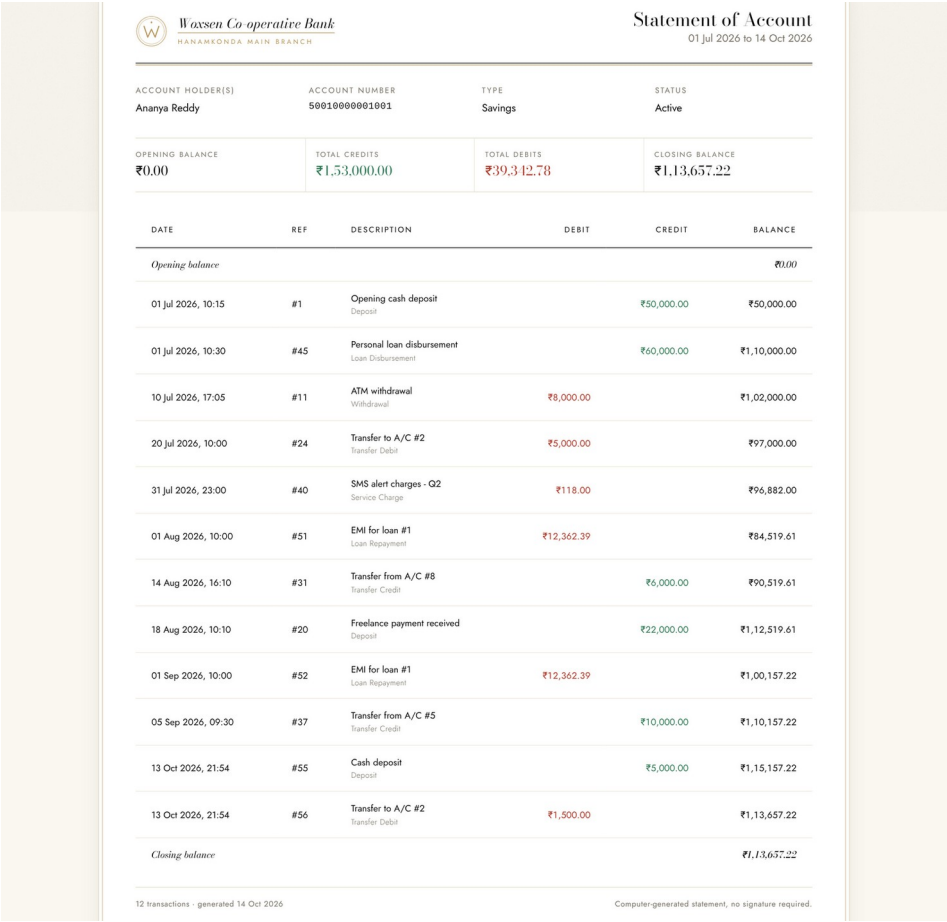


Figure 9. Loan Schedule After an EMI Repayment (Update)



Waxsen Co-operative Bank
HANAMKONDA MAIN BRANCH

Statement of Account
01 Jul 2026 to 14 Oct 2026

ACCOUNT HOLDER(S)	ACCOUNT NUMBER	TYPE	STATUS
Ananya Reddy	50010000001001	Savings	Active

OPENING BALANCE	TOTAL CREDITS	TOTAL DEBITS	CLOSING BALANCE
₹0.00	₹1,53,000.00	₹39,342.78	₹1,13,657.22

DATE	REF	DESCRIPTION	DEBIT	CREDIT	BALANCE
Opening balance					*₹0.00*
01 Jul 2026, 10:15	#1	Opening cash deposit Deposit		₹50,000.00	₹50,000.00
01 Jul 2026, 10:30	#45	Personal loan disbursement Loan Disbursement		₹60,000.00	₹1,10,000.00
10 Jul 2026, 17:05	#11	ATM withdrawal Withdrawal	₹8,000.00		₹1,02,000.00
20 Jul 2026, 10:00	#24	Transfer to A/C #2 Transfer Debit	₹5,000.00		₹97,000.00
31 Jul 2026, 23:00	#40	SMS alert charges - Q2 Service Charge	₹118.00		₹96,882.00
01 Aug 2026, 10:00	#51	EMI for loan #1 Loan Repayment	₹12,362.39		₹84,519.61
14 Aug 2026, 16:10	#31	Transfer from A/C #8 Transfer Credit		₹6,000.00	₹90,519.61
18 Aug 2026, 10:10	#20	Freelance payment received Deposit		₹22,000.00	₹1,12,519.61
01 Sep 2026, 10:00	#52	EMI for loan #1 Loan Repayment	₹12,362.39		₹1,00,157.22
05 Sep 2026, 09:30	#37	Transfer from A/C #5 Transfer Credit		₹10,000.00	₹1,10,157.22
13 Oct 2026, 21:54	#55	Cash deposit Deposit		₹5,000.00	₹1,15,157.22
13 Oct 2026, 21:54	#56	Transfer to A/C #2 Transfer Debit	₹1,500.00		₹1,13,657.22
Closing balance					*₹1,13,657.22*

12 transactions · generated 14 Oct 2026

Computer-generated statement, no signature required.

Figure 10. Account Statement Report with Running Balance

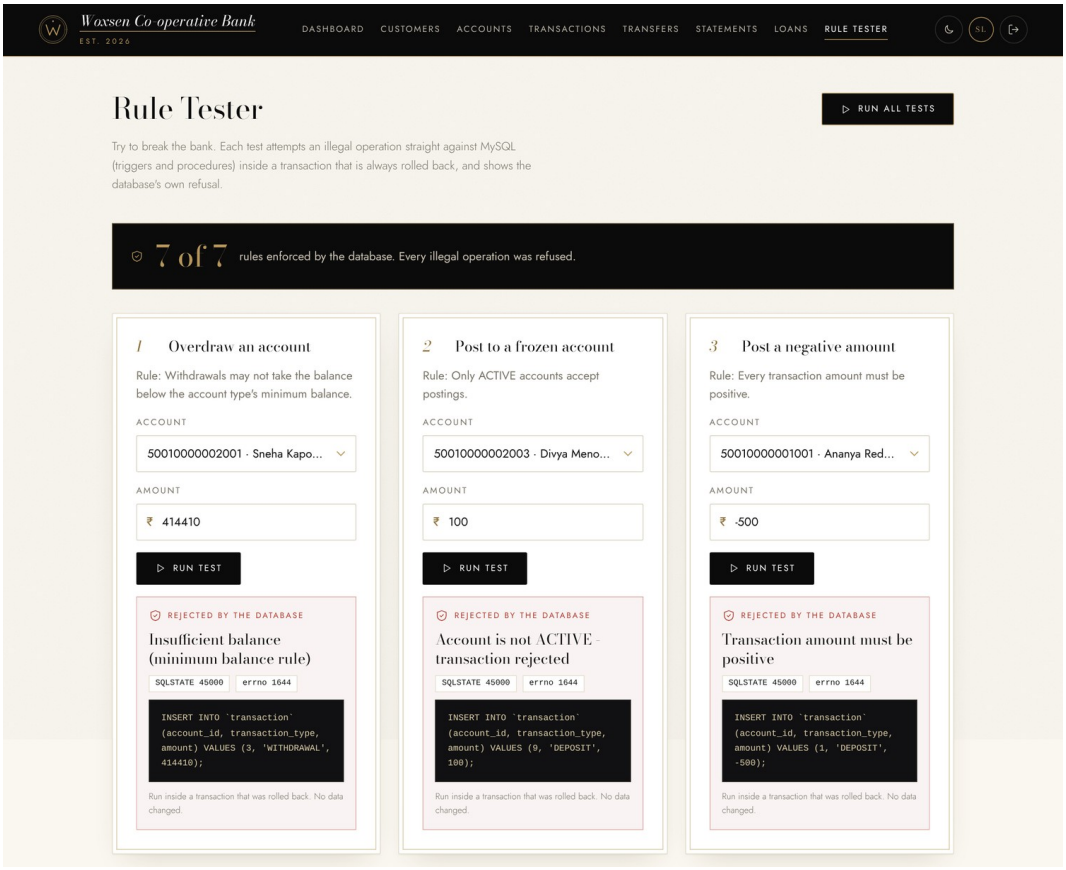


Figure 11. Rule Tester: Illegal Operations Refused and Rolled Back

3.8 Testing and Results

Table 15 lists 16 test cases that were executed against the running application and the live database. The cases mix positive and negative inputs; actual results are the messages and values observed.

Table 15. Test Cases

Test ID	Feature	Input / Action	Expected Result	Actual Result	Status
TC-01	Staff login	Sign in as teller with the correct password	Session starts; dashboard opens	HTTP 200; dashboard loaded for role TELLER	Pass
TC-02	Staff login (negative)	Sign in with a wrong password	Login refused	HTTP 401 'Incorrect username or password'	Pass
TC-03	Customer search	Search customers for 'ananya'	Only matching customer returned	1 row: Ananya Reddy, 1 account	Pass
TC-04	Deposit	Deposit INR 5,000 to account 50010000001001	Posting created; balance increases by 5,000	Transaction #55; balance 1,10,157.22 → 1,15,157.22	Pass
TC-05	Overdraw (negative)	Withdraw INR 99,99,999 from account 50010000004001	Rejected; balance unchanged	422 'Insufficient balance (minimum balance rule)'; balance stays 89,000.00	Pass
TC-06	Negative amount	Deposit INR -500	Rejected	422 'Transaction amount must be positive'	Pass
TC-07	Frozen account	Deposit INR 1,000 to FROZEN account 9	Rejected	422 'Account is not ACTIVE - transaction rejected'	Pass
TC-08	Closed account	Deposit INR 1,000 to CLOSED account 11	Rejected	422 'Account is not ACTIVE - transaction rejected'	Pass

Test ID	Feature	Input / Action	Expected Result	Actual Result	Status
TC-09	Transfer	Transfer INR 1,500 from ...1001 to ...1002	Debit = credit; both balances move	Transfer #9 balanced; 1,15,157.22 → 1,13,657.22 and 73,764.00 → 75,264.00	Pass
TC-10	Same-account transfer	Transfer INR 100 from account 1 to account 1	Rejected	422 'Cannot transfer to the same account'	Pass
TC-11	Oversized transfer	Transfer INR 9,99,99,999 from a small account	Rejected and rolled back; no half transfer	422 'Insufficient balance (minimum balance rule)'; no rows added	Pass
TC-12	Loan repayment	Repay EMI INR 12,362.39 of loan #1	Instalment PAID; account debited; outstanding falls	200; outstanding 37,087.17 → 24,724.78 (after the 3rd EMI); account debited by 12,362.39	Pass
TC-13	Repay paid instalment	Repay an instalment already PAID	Refused	409 'This instalment is already fully paid'	Pass
TC-14	Statement	Statement of 50010000001001, 01 Jul to 14 Oct 2026; then an invalid range	Opening, closing and running balance; bad range refused	Opening 0.00, credits 1,53,000.00, debits 39,342.78, closing 1,13,657.22; reversed dates: 422 ck_statement_period	Pass
TC-15	Role enforcement	Teller creates a loan; loan officer posts a deposit; teller opens audit log	All refused	HTTP 403 'Your role (...) is not allowed to do this' in all three	Pass
TC-16	Ledger integrity	Run balance-versus-ledger audit; 6 parallel withdrawals; Rule Tester 'Run all'	0 mismatches; no overdraw; 7 of 7 refused	0 rows mismatched; 0 unbalanced transfers; parallel-withdrawal test passed; 7 of 7 refused	Pass

All 16 test cases passed. Positive cases (TC-01, 03, 04, 09, 12, 14) changed the database as expected, and every negative case (TC-02, 05 to 08, 10, 11, 13, 15) was refused with the exact message of the rule it breaks while balances stayed unchanged. Figure 12 shows the same behaviour directly in MySQL, outside the application: the trigger rejects the overdraw and the frozen-account deposit and the balances are unchanged.

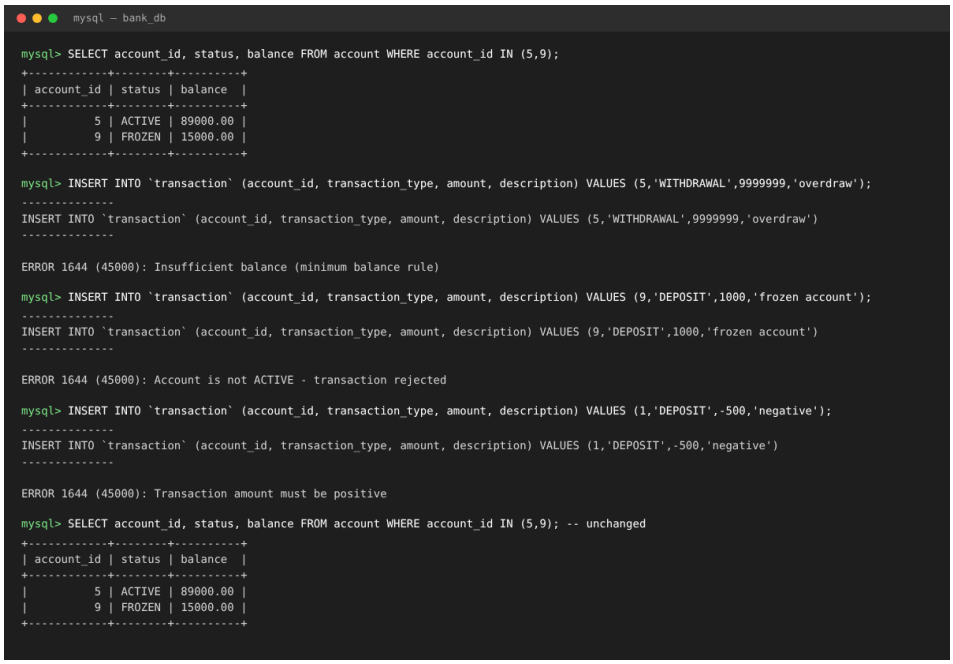


Figure 12. Trigger Rejection and Unchanged Balances in MySQL

Besides these cases, an automated end-to-end suite of 15 tests (npm test) runs against the real database. It passed 15 of 15 on a freshly loaded database; it covers atomic rollback of failed transfers and loans, role enforcement, loan closure after the last instalment, and parallel withdrawals that never overdraw an account. The suite assumes a freshly loaded database, so it is run after reloading the script. The application therefore uses the designed database correctly and enforces the important business rules in the database itself.

CHAPTER 4

CONCLUSION

This project addressed the problem of keeping bank accounts and transactions correct, consistent and auditable. An entity-relationship model with 14 entities was converted into a 3NF relational schema with 17 foreign keys, 11 unique and 11 check constraints, and implemented in MySQL with 3 triggers, 2 stored procedures, 7 views and 5 indexes. The business rules (positive amounts, minimum balance, active accounts only, balanced transfers, one payment per instalment) live in the database, so no client can bypass them.

On top of this design a role-based Node.js and Express application was built for customers, accounts, deposits, withdrawals, transfers, loans, repayments, statements and reports, and it was validated with 16 test cases and an automated suite. The main learning was that placing rules in triggers and procedures, and running multi-step operations as transactions, gives stronger guarantees than validating in the application alone.

4.1 Limitations and Future Enhancements

The current application is deliberately small in scope. Its main limitations and realistic enhancements are:

- Limitation: interest on savings and fixed deposits is not posted automatically; a scheduled job could credit interest and apply minimum-balance penalties each month.
- Limitation: transfers to external banks can be saved as beneficiaries but are not settled; integration with a payment network (for example UPI or NEFT) would be needed for real settlement.
- Enhancement: add customer self-service login with one-time passwords, e-mail or SMS alerts, and downloadable statements in PDF.
- Enhancement: add table partitioning of the transaction ledger by month and reporting for fraud detection on large or unusual transfers.

REFERENCES

[1] E. F. Codd, “A relational model of data for large shared data banks,” *Commun. ACM*, vol. 13, no. 6, pp. 377-387, Jun. 1970, doi: 10.1145/362384.362685.

[2] P. P.-S. Chen, “The entity-relationship model—toward a unified view of data,” *ACM Trans. Database Syst.*, vol. 1, no. 1, pp. 9-36, Mar. 1976, doi: 10.1145/320434.320440.

[3] R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th ed. Hoboken, NJ, USA: Pearson, 2016.

[4] A. Silberschatz, H. F. Korth, and S. Sudarshan, *Database System Concepts*, 7th ed. New York, NY, USA: McGraw-Hill, 2020.

[5] Oracle Corporation, “MySQL 8.0 Reference Manual: SIGNAL Statement,” MySQL Documentation. Accessed: Oct. 14, 2026. [Online]. Available: https://dev.mysql.com/doc/refman/8.0/en/signal.html

[6] OpenJS Foundation, “Routing,” Express.js Documentation. Accessed: Oct. 14, 2026. [Online]. Available: https://expressjs.com/en/guide/routing.html

[7] M. Jones, J. Bradley, and N. Sakimura, “JSON Web Token (JWT),” RFC 7519, Internet Engineering Task Force, May 2015. [Online]. Available: https://www.rfc-editor.org/rfc/rfc7519

[8] OWASP Foundation, “SQL Injection Prevention Cheat Sheet,” OWASP Cheat Sheet Series. Accessed: Oct. 14, 2026. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html

APPENDICES

APPENDIX A - COMPLETE DATABASE SCRIPT

The complete database script (DDL, triggers, stored procedures, sample data, views and queries) is submitted as bank_review2.sql and, split by purpose, in the repository folder review-2/sql/ (01_schema.sql, 02_triggers.sql, 03_sample_data.sql, 04_views_reports.sql, 05_queries.sql). Run order: 01 to 04, then 05. The expected output of the queries is in queries_output.txt, and the seven validation tests are in validation_tests.md.

APPENDIX B - APPLICATION SOURCE CODE

The application source code is in the GitHub repository https://github.com/srilakshmireddy-byte/bank-dbms-pbl, folder review-3/ (server.js, db.js, routes/, lib/, middleware/, public/, test/). A live demonstration that runs in the browser on built-in sample data is available at https://srilakshmireddy-byte.github.io/bank-dbms-pbl/. To run the full MySQL-backed version: load the SQL script, copy .env.example to .env, run npm install, then npm start (default port 3000). Demo logins: teller, loans and manager.

APPENDIX C - ADDITIONAL TEST DATA AND OUTPUTS

Supporting output. The seven validation tests below were run directly in MySQL; each statement was rejected by the database and the balances stayed unchanged.

#	Test	Result (actual MySQL error)
1	Negative amount deposit	Transaction amount must be positive
2	Withdrawal larger than balance	Insufficient balance (minimum balance rule)
3	Deposit to FROZEN account	Account is not ACTIVE - transaction rejected
4	Deposit to CLOSED account	Account is not ACTIVE - transaction rejected
5	Transfer to the same account	Cannot transfer to the same account
6	Transfer larger than balance (rolled back)	Insufficient balance (minimum balance rule)
7	Duplicate account number	Duplicate entry '50010000001001' for key 'account_number'

Consistency checks after the sample load: 0 accounts whose stored balance differs from the last ledger balance; 0 transfers whose debited amount differs from the credited amount; 54 ledger transactions loaded.